Частное общеобразовательное учреждение «РЖД лицей №13»

Кейс «Обучение через технологии»

Интерактивный тренажер «DIV, MOD и порядок действий в арифметическом выражении»

Авторы:
обучающиеся 10 класса
Щенников Олег Никитич
Володин Михаил Сергеевич
Руководители:
учителя информатики
Волгина Маргарита Петровна
Руденко Ирина Сергеевна

Цель и задачи

- Цель:

  Создать удобный и наглядный тренажер для автоматизации навыков работы с операторами целочисленного деления DIV и MOD, а также для понимания приоритета операций в составных выражениях.

- Задачи:

  1. Разработать три режима: «DIV», «MOD» и «Порядок действий»

  2. Реализовать мгновенную проверку ответов с пояснениями.

  3. Обеспечить адаптивный интерфейс для ПК, планшетов и смартфонов.

  4. Создать систему поощрения (переход на следующий уровень при правильном ответе).

Общий вид главного меню

При запуске файла html пользователь может выбрать 1 из трех режимов:

1 — Раздел «DIV»

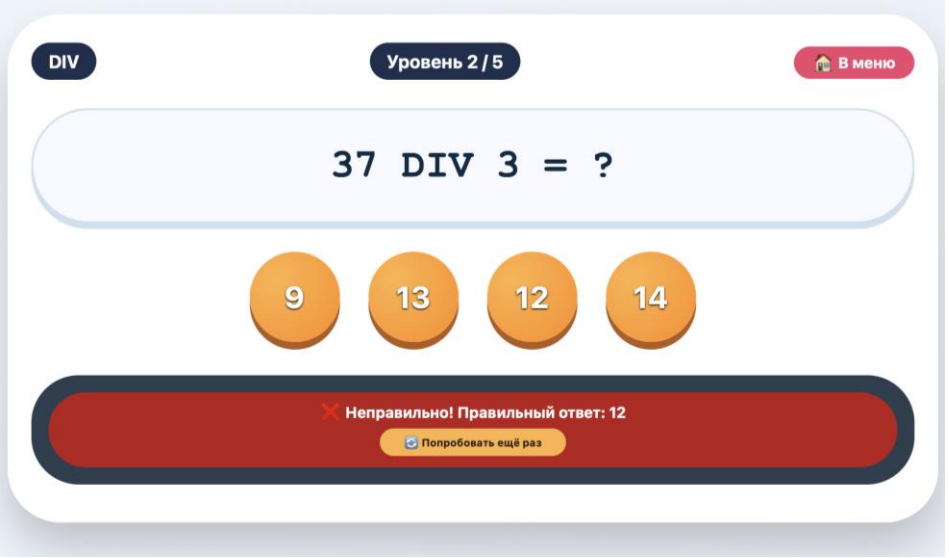
2 — Раздел «MOD»

3 — Раздел «Порядок действий»

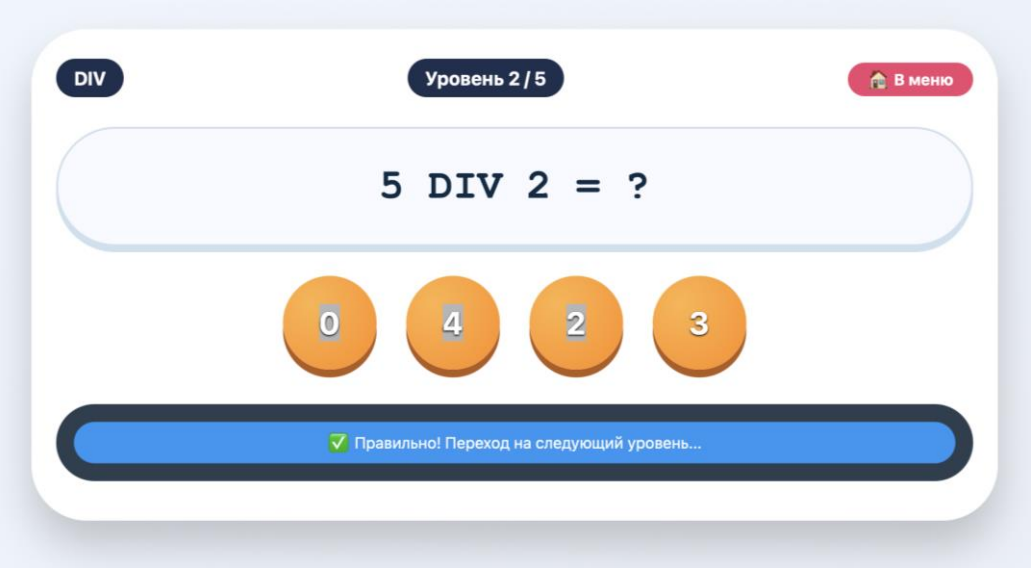
Также в нижней части экрана есть подсказка для пользователя(4)

Режимы «DIV» и «MOD» - принцип работы

- Генератор случайных чисел создаёт пример на основе текущего уровня (1–5).
- Чем выше уровень, тем больше диапазон чисел.
- Пользователь видит 4 варианта ответа в виде крупных кругов.
- При нажатии на круг:
  - Правильный ответ → переход на следующий уровень.
  - Неправильный ответ → показывается правильный ответ и кнопка «Попробовать ещё раз».



Обратная связь и переходы между заданиями



Принцип проверки:

- После ввода всех значений программа сравнивает каждую переменную с эталоном.
- Если всё верно → зелёное сообщение, через 1 секунду автоматический переход к следующему примеру.
- Если ошибка → красное сообщение с правильными ответами, пользователь может исправить ввод и нажать «Проверить» снова.
- После последнего (5-го) примера – поздравление и возврат в главное меню.

Генерация вопросов и вариантов ответа — математическая логика

```
// ------------------- ГЕНЕРАТОР ВОПРОСОВ ДЛЯ DIV/MOD -------------------
function generateQuestion(operation, level) {
    const maxDivisor = Math.min(12, 6 + level);
    const minDivisor = 2;
    const divisor = Math.floor(Math.random() * (maxDivisor - minDivisor + 1) + minDivisor);
    let dividend, correct;
    if (operation === 'div') {
        const quotient = Math.floor(Math.random() * (8 + level * 2) + 2);
        const remainder = Math.floor(Math.random() * divisor);
        dividend = divisor * quotient + remainder;
        correct = Math.floor(dividend / divisor);
    } else {
        const minDividend = divisor * 3;
        const maxDividend = divisor * 8 + level * 5;          const minDividend: number
        dividend = Math.floor(Math.random() * (maxDividend - minDividend + 1) + minDividend);
        correct = dividend % divisor;
    }
    const correctAnswer = correct;
    let wrongOptions = new Set();
    let attempts = 0;
    while (wrongOptions.size < 3 && attempts < 20) {
        let offset;
        if (operation === 'div') {
            offset = Math.floor(Math.random() * 7) - 3;
            let cand = correctAnswer + offset;
            if (cand < 0) cand = correctAnswer + 1;
            if (cand !== correctAnswer) wrongOptions.add(cand);
        } else {
            let shift = Math.floor(Math.random() * (divisor - 1)) + 1;
            let cand = (correctAnswer + shift) % divisor;
            if (cand !== correctAnswer) wrongOptions.add(cand);
            let cand2 = (correctAnswer - shift + divisor) % divisor;
            if (cand2 !== correctAnswer && !wrongOptions.has(cand2)) wrongOptions.add(cand2);
        }
    }
```

```
    let wrongArr = Array.from(wrongOptions).slice(0, 3);
    while (wrongArr.length < 3) {
        wrongArr.push(correctAnswer + (wrongArr.length + 1));
    }
    let options = [correctAnswer, ...wrongArr];
    for (let i = options.length - 1; i > 0; i--) {
        const j = Math.floor(Math.random() * (i + 1));
        [options[i], options[j]] = [options[j], options[i]];
    }
    let questionText = (operation === 'div')
        ? `${dividend} DIV ${divisor} = ?`
        : `${dividend} MOD ${divisor} = ?`;
    return { text: questionText, correct: correctAnswer, options: options.slice(0, 4) };
}
```

Метод создания отвлекающих ответов:

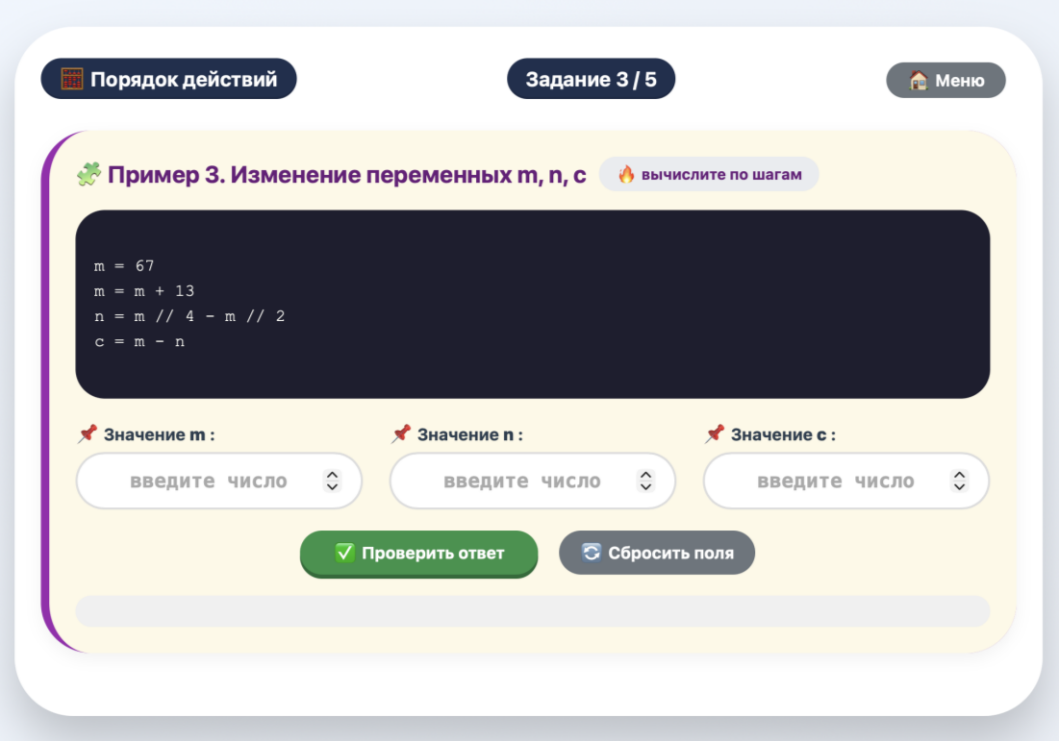
· Для DIV: правильное частное смещается на ±1, ±2, ±3, но остаётся неотрицательным.

· Для MOD: генерируются смещения по модулю делителя (например, для делителя 5 неправильные варианты → 0,1,2,3,4, исключая верный остаток).

Почему это эффективно:

Ученик не может просто угадать – все варианты выглядят правдоподобно, но ошибочные ответы помогают закрепить правило.

Режим «Порядок действий» – структура



Описание режима:

- Пользователь видит блок кода с последовательными операторами присваивания.
- Нужно мысленно (или на бумаге) выполнить все действия, соблюдая приоритет операций (порядок действий).
- Затем ввести получившиеся значения переменных в текстовые поля.
- После нажатия «Проверить ответ» программа сравнивает введённые числа с эталонными.

Фронтенд (чистый HTML/CSS/JS без библиотек):

- HTML5 – семантическая разметка, адаптивные контейнеры.

```html
<div class="game-container" id="app">
```

- CSS3 – Flexbox для сеток, @media для мобильных устройств, box-shadow для объёмных кнопок, user-select: none для защиты от случайного выделения.

```css
@media (max-width: 600px) {
    .game-container { padding: 20px; }
    .circle { width: 70px; height: 70px; font-size: 1.5rem; }
    .question-card { font-size: 1.5rem; }
    .mode-btn { width: 140px; font-size: 1.4rem; }
    .code-block { font-size: 0.8rem; }
    .input-field input { font-size: 1rem; padding: 10px; }
```

- JavaScript (ES6) – управление состояниями (gameState, currentTaskIndex), динамическая генерация DOM-элементов, обработка событий.

```javascript
let gameState = {
    operation: null,
    level: 1,
    currentQuestion: null,
    awaitingResponse: true,
    errorTimeout: null
};
```

Алгоритмические методы:

- Генератор псевдослучайных примеров с учётом уровня.

(См. Слайд 6)

- Проверка на уникальность неправильных ответов (использование Set).

(См. Слайд 6)

- Таймеры (setTimeout) для плавных переходов после правильного ответа.

```
setTimeout(() => loadNextLevel(), 800);
```

- Машина состояний – переключение между экранами (меню, игра, задания).

```
Div.classList.add('hide');
```

UI/UX решения:

Анимация нажатия (transform: translateY, изменение box-shadow).

```
transform: translateY(4px);
```

Цветовое кодирование: зелёный – DIV, оранжевый – MOD, фиолетовый – задания на порядок действий.

```
.mode-btn.div {
    background: #2b9348;
```

```
.mode-btn.mod {
    background: #e67e22;
```

```
.mode-btn.challenge {
    background: #9c27b0;
```

Крупные интерактивные элементы для касания пальцем.

```
.mode-btn {
    background: #4c6ef5;
    border: none;
    width: 170px;
    padding: 20px 0;
    font-size: 1.8rem;
    font-weight: bold;
    color: white;
    border-radius: 70px;
    cursor: pointer;
    transition: all 0.25s;
    box-shadow: 0 8px 0 #2c3e8f;
    font-family: monospace;
    letter-spacing: 2px;
}
```

Адаптивность и кросс-браузерность

Что было сделано:

- Использованы относительные единицы (%, vw, rem).
- Медиа-запрос для экранов ≤600px уменьшает размер кругов, шрифтов и отступов.
- Все кнопки имеют достаточную область нажатия (44×44 пикселя минимум).
- Гибкое расположение элементов через flex-wrap.
- Тестирование: Chrome, Firefox, Safari (macOS/iOS), Edge.

Итоги и результаты

Что удалось реализовать:

- 3 полноценных режима обучения.
- 5 уровней сложности для DIV/MOD.
- 5 комплексных заданий на порядок действий.
- Адаптивный дизайн.
- Генерацию правдоподобных неправильных вариантов.

Объём кода: ~750 строк чистого HTML/CSS/JS (без внешних библиотек).

Практическая ценность: Тренажёр можно использовать на уроках информатики, для самостоятельной подготовки к контрольным работам и для развития алгоритмического мышления.

Дальнейшее развитие

Планы на улучшение:

- Добавить режим «Соревнование» (таймер и подсчёт очков).
- Возможность создавать свои примеры через текстовое поле.
- Статистику правильных/неправильных ответов за сессию.
- Экспорт результатов в PDF.

Спасибо за внимание!

Ссылка на репозиторий с исходным кодом:
https://github.com/OJlezKal7eJlbMeLLlKa/DivModPermTrainer